Malicious Code Analysis : CMU Heinz College 95-759
Lab #3 : Signature Development

Name: Brendan R.
Date: 25 February 2026

Challenges:

Section 1 : SFX RAR Detection

For this section, you will need to study various file formats to understand their file 'magic'. The file magic is usually a sequence of identifiable bytes at the beginning of a file software initially uses to test if the file is properly formatted.

1. In the previous Lab, you were asked to review strings that were indicative of a SFX RAR file. Use these strings to create a signature you feel confident will reliably detect on an SFX RAR and paste your signature below along with a screenshot of a successful detection.
    a. Signature

```
blurt@blurt-ubuntu-mca:~/Downloads/Lab3$ head -n 15 resume_NoTrigger.yara
rule resume_MCA {

        strings:
                $magic = { 61 52 21 72 }
                $magic2 = { 52 61 72 21 }
                // $pe = { 4D 5A }
                //$comment = ";The comment below contains SFX script commands"
                 $comment2 = "ui37893uibfc7ab3jnekl78wg2"
                // $visualbasic = "test.vbs"
                // $Setup = "nc.exe"

        condition:
                ($magic or $magic2) and $comment2
}
blurt@blurt-ubuntu-mca:~/Downloads/Lab3$ yara -s resume_NoTrigger.yara resume.zip.exe
blurt@blurt-ubuntu-mca:~/Downloads/Lab3$ head -n 15 resume_Trigger.yara
rule resume_MCA {

        strings:
                $magic = { 61 52 21 72 }
                $magic2 = { 52 61 72 21 }
                // $pe = { 4D 5A }
                $comment = ";The comment below contains SFX script commands"
                // $comment2 = "ui37893uibfc7ab3jnekl78wg2"
                // $visualbasic = "test.vbs"
                // $Setup = "nc.exe"

        condition:
                ($magic or $magic2) and $comment
}
blurt@blurt-ubuntu-mca:~/Downloads/Lab3$ yara -s resume_Trigger.yara resume.zip.exe
resume_MCA resume.zip.exe
0x72c00:$magic2: 52 61 72 21
0x72c30:$comment: ;The comment below contains SFX script commands
blurt@blurt-ubuntu-mca:~/Downloads/Lab3$
```

2. Scan your own signature file with your signature. Does it successfully detect? Why or why not? What can be done to increase the fidelity of your signature to reduce false positives (*If your signature does not produce a false positive, I still expect you to identify opportunities to improve it! Hint: what is the file format?)*?

```
blurt@blurt-ubuntu-mca:~/Downloads/Lab3$ yara -s resume_Trigger.yara resume_Trigger.yara
blurt@blurt-ubuntu-mca:~/Downloads/Lab3$ yara -s resume_SelfTrigger.yara resume_SelfTrigger.yara
resume_MCA resume_SelfTrigger.yara
0x7e:$comment: ;The comment below contains SFX script commands
0xef:$visualbasic: test.vbs
0x105:$setup: nc.exe
blurt@blurt-ubuntu-mca:~/Downloads/Lab3$
```

Rule from Q1 does not self-detect because it is searching for the magic SFX RAR byte sequence which a .yara file does not have. Rewrote the rule to self trigger on strings only, which the .yara file does contain, so it false-posisitive triggers.

Section 2 : YARA + Python API

One of the powerful features of YARA is the ability to combine its pattern matching features with other programming languages. In this section we will focus on using the above-mentioned signature to trigger an action.

1. Use your previous signature to write a Python script using the YARA module to automatically extract the embedded RAR file when the signature detects it. Write the extracted files to disk. Copy and paste your script below along with a screenshot of it running and a file listing of the produced data.

```
import yara
import rarfile
import os
import sys
import io

TARGET_FILE   = "resume.zip.exe"
YARA_RULE     = "resume_Trigger.yara"
OUTPUT_DIR    = "extracted"

RAR4_MAGIC = b"\x52\x61\x72\x21\x1a\x07\x00"

def offset(data):
    for magic in (RAR4_MAGIC):
        offset = data.find(magic)
        if offset != -1:
            return offset
    return -1

def extract_rar(data, rar_offset, output_dir):
    rar_data = data[rar_offset:]
    rar_buffer = io.BytesIO(rar_data)
```

```
    with rarfile.RarFile(rar_buffer) as rf:
        contents = rf.namelist()

        for item in contents:
            rf.extract(item, path=output_dir)
            out_path = os.path.join(output_dir, item)
            size = os.path.getsize(out_path)

def main():

    with open(TARGET_FILE, "rb") as f:
        data = f.read()

    rar_offset = offset(data)

    extract_rar(data, rar_offset, OUTPUT_DIR)

if __name__ == "__main__":
    main()
```

```
blurt@blurt-ubuntu-mca:~/Downloads/Lab3/extracted$ ls
resume.zip.exe  yara_extracted1.py
blurt@blurt-ubuntu-mca:~/Downloads/Lab3/extracted$ python3 yara_extracted1.py
blurt@blurt-ubuntu-mca:~/Downloads/Lab3/extracted$ ls
extracted  resume.zip.exe  yara_extracted1.py
blurt@blurt-ubuntu-mca:~/Downloads/Lab3/extracted$ cd extracted/
blurt@blurt-ubuntu-mca:~/Downloads/Lab3/extracted/extracted$ ls
nc.exe  resume.docx  test.vbs
blurt@blurt-ubuntu-mca:~/Downloads/Lab3/extracted/extracted$
```

2. Now use a signature designed to detect executables (you may use the one from lecture or your own) to write another python script. Using the YARA module, write a Python script that, when alerting on an executable, tells the user the following information. (*Hint: You may find it useful to look at the pefile Python module as well as Microsoft documentation concerning the PE file format*).
    a. The file name
    b. The file size
    c. The MD5 hash of the file
    d. Determine if ASLR enabled
    e. Determine if DEP enabled
    f. If the file is an executable or a DLL

    Copy and paste your script below, along with a screenshot of it running and producing the above data.

```
def extract_rar(data, rar_offset, output_dir):
    rar_data = data[rar_offset:]
    rar_buffer = io.BytesIO(rar_data)

    with rarfile.RarFile(rar_buffer) as rf:
        contents = rf.namelist()

        for item in contents:
            rf.extract(item, path=output_dir)
            out_path = os.path.join(output_dir, item)
            size = os.path.getsize(out_path)

def scan_extracted():
    rules = yara.compile(source=YARA_RULE_SOURCE)

    for filename in os.listdir(OUTPUT_DIR):
        filepath = os.path.join(OUTPUT_DIR, filename)

        matches = rules.match(filepath)

        if matches:
            with open(filepath, "rb") as f:
                data = f.read()

            pe = pefile.PE(filepath)

            md5 = hashlib.md5(data).hexdigest()
            size = os.path.getsize(filepath)
            dll_chars = pe.OPTIONAL_HEADER.DllCharacteristics
            file_chars = pe.FILE_HEADER.Characteristics

            aslr = bool(dll_chars & 0x0040)
            dep  = bool(dll_chars & 0x0100)
            is_dll = bool(file_chars & 0x2000)

            print("File Name :", filename)
            print("File Size :", size, "bytes")
            print("MD5       :", md5)
            print("ASLR      :", "Enabled" if aslr else "Disabled")
            print("DEP       :", "Enabled" if dep else "Disabled")
            print("Type      :", "DLL" if is_dll else "EXE")
            print("")

def main():

    with open(TARGET_FILE, "rb") as f:
        data = f.read()

    rar_offset = offset(data)
```

```
    extract_rar(data, rar_offset, OUTPUT_DIR)

    scan_extracted()

if __name__ == "__main__":
    main()
```